

SymPy Bug Report

1 Bug 45: Missing Real Poles for a Principal Fractional-Power Denominator

1.1 Minimal Reproducer

```
from sympy import Rational, S, symbols
from sympy.calculus.singularities import singularities

x = symbols("x")
expr = 1/((x**2)**Rational(1, 3) - 1)
s = singularities(expr, x, S.Reals)

print("singularities(...) =", s)
print("value at x = 1:", expr.subs(x, 1))
print("value at x = -1:", expr.subs(x, -1))
```

1.2 Observed Behavior

```
SymPy version: 1.14.0
singularities(1/((x**2)**Rational(1, 3) - 1), x, S.Reals) = EmptySet
value at x = 1: zoo
value at x = -1: zoo
```

SymPy reports no real singularities for

$$\frac{1}{(x^2)^{1/3} - 1}.$$

This is wrong because direct substitution at $x = 1$ and $x = -1$ gives complex infinity, indicating poles of the reciprocal expression.

1.3 Expected Behavior

The real singularity set should include both -1 and 1 . For this representative expression, the mathematically correct real singularities are

$$\{-1, 1\}.$$

1.4 Mathematical Explanation

The denominator vanishes exactly when

$$(x^2)^{1/3} - 1 = 0.$$

For real x , $x^2 \geq 0$, so $(x^2)^{1/3}$ is the ordinary nonnegative real cube root of x^2 . Therefore

$$(x^2)^{1/3} = 1 \iff x^2 = 1 \iff x \in \{-1, 1\}.$$

At both points the denominator is zero and the reciprocal has a pole:

$$\frac{1}{(1^2)^{1/3} - 1} = \frac{1}{0}, \quad \frac{1}{((-1)^2)^{1/3} - 1} = \frac{1}{0}.$$

Thus ‘EmptySet’ is not an equivalent representation, a harmless branch choice, or a removable omission. It excludes two real points where the expression is actually singular.

1.5 Source-Level Diagnosis

The diagnosis identifies the singularity finder in `sympy/calculus/singularities.py`. It also identifies the equation-solving logic in `sympy/solvers/solveset.py`. The traced call path is that `singularities` rewrites the reciprocal and, for the denominator, calls

```
solveset((x**2)**Rational(1, 3) - 1, x, S.Reals).
```

The public `solveset` wrapper replaces the unconstrained symbol with a real dummy. Along that path, the equation is rewritten as

$$|_R|^{2/3} - 1 = 0.$$

The traced internal inversion is

```
_invert(Abs(_R)**(2/3) - 1, 0, _R, S.Reals) → (_R, EmptySet).
```

The immediate problematic rule is in `_invert_abs` around `sympy/solvers/solveset.py:612-617`. The preceding inversion of $|_R|^{2/3} = 1$ produces the finite target set $\{-1, 1\}$ for $|_R|$. The target -1 is invalid for an absolute value, but the target 1 is valid and should lead to $_R \in \{-1, 1\}$. Instead, `_invert_abs` rejects the whole finite target set as soon as any target is negative:

```
elif not ok:
    return symbol, S.EmptySet
```

This is too strong for mixed finite target sets. It discards the valid $\text{Abs}(_R) = 1$ branch along with the impossible $\text{Abs}(_R) = -1$ branch. Since `singularities` relies on `solveset` to find denominator zeros, the discarded branch leaves it with no points to add and it returns `EmptySet`. The diagnosis supports a plausible fix direction: for finite target sets, `_invert_abs` should filter to nonnegative targets and return `EmptySet` only if no nonnegative targets remain, while preserving the existing conditional handling for targets whose sign is not known.

1.6 Additional Failing Instances

The same failure appears for shifted denominators

$$f_a(x) = \frac{1}{(((x-a)^2)^{1/3} - 1)}, \quad a \in \{0, 1, 2, 3, 4, 5\}.$$

For each a , the denominator vanishes when $(x-a)^2 = 1$, so the expected real poles are $x = a - 1$ and $x = a + 1$. The independent parametrized check verifies that SymPy's returned singularity set does not contain these required points.

Input / Expression	SymPy behavior	Expected behavior
$1/(((x-0)^2)^{1/3} - 1)$	Omits the poles -1 and 1 .	Contains $\{-1, 1\}$.
$1/(((x-1)^2)^{1/3} - 1)$	Omits the poles 0 and 2 .	Contains $\{0, 2\}$.
$1/(((x-2)^2)^{1/3} - 1)$	Omits the poles 1 and 3 .	Contains $\{1, 3\}$.
$1/(((x-3)^2)^{1/3} - 1)$	Omits the poles 2 and 4 .	Contains $\{2, 4\}$.
$1/(((x-4)^2)^{1/3} - 1)$	Omits the poles 3 and 5 .	Contains $\{3, 5\}$.
$1/(((x-5)^2)^{1/3} - 1)$	Omits the poles 4 and 6 .	Contains $\{4, 6\}$.

The expected behavior is uniform across the family because the inner square is nonnegative for real x . Thus the principal fractional power introduces no negative-input branch ambiguity at the actual denominator zeros; every listed case reduces to the real equation $(x-a)^2 = 1$.

1.7 Related Issues Assessment

The best-effort deduplication check found public material documenting **singularities** and branch-sensitive principal powers, as well as general CAS-family discussion of fractional-power simplification hazards. It did not identify a clear public duplicate for the exact failure shown above: the representative **singularities** call returns `EmptySet`. The deduplication verdict is that this is a new instance of a known failure mode: the broad fractional-power/branch-sensitivity issue is known, but this specific real singularity miss is previously unreported in the available evidence and remains individually actionable as an issue and regression test.

1.8 Proposed SymPy Regression Test

```
import pytest
from sympy import Rational, S, symbols
from sympy.calculus.singularities import singularities

@pytest.mark.parametrize("a", [0, 1, 2, 3, 4, 5])
def test_singularities_include_poles_of_shifted_principal_power(a):
    x = symbols("x")
    expr = 1/(((x - a)**2)**Rational(1, 3) - 1)
    s = singularities(expr, x, S.Reals)
    assert s.contains(a - 1) is S.true
    assert s.contains(a + 1) is S.true
```